📡 PROJECT BIBLE

Global Compliance Rada

A complete, plain-English guide to everything this sys
and how every piece of code connects. Written so tha
— can pick this up and understand the whole thing fr

Purpose of This Document

This document is the single source of truth for the Global C
system does, why every technology decision was made,
codebase does.

Anyone who picks this up — a new developer, a product ma
member — should be able to read it from top to bottom and
needing to dig through the raw code.

> **WHO SHOULD READ THIS**
>
> New engineers onboarding to the project, compliance stake
> makes decisions, anyone extending the system with new fe
> production.

What Is This Project?

The Global Compliance Radar is an **automated intelligence**
financial regulations, processes them with artificial intellige
briefing dashboard.

Every day, without anyone pressing a button, the system wa
announcements from six major global financial regulators, s
summaries into a database. A web dashboard then displays
can see what changed overnight.

The problem it solves

Financial institutions must track AML/CFT (Anti-Money Lau
regulations from dozens of regulators across the US, EU, U
compliance analysts had to manually visit each regulator's
decide what was relevant. That process is slow, inconsister
automates it entirely.

Who Is It For?

Compliance Analyst / AML Officer

Needs a daily, prioritised digest of regulatory changes without manually scanning ten websites every morning. Uses the dashboard to filter by jurisdiction and relevance score.

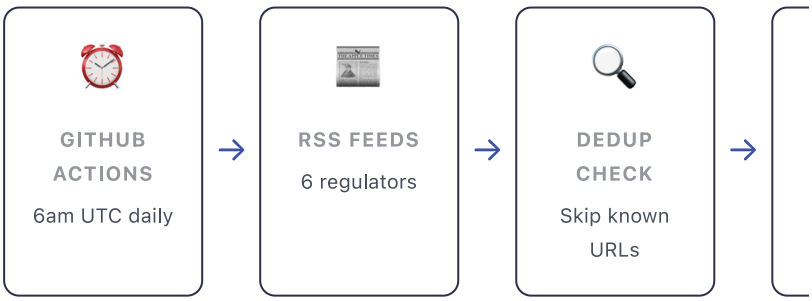
New Developer

Joining the project and needs to understand the architecture before making changes. This document is written for them.

The Big Picture

The system has two completely separate sides that share c
intentional: the data pipeline only writes, and the UI only re

The daily pipeline (write side)



The dashboard (read side)



Technology Stack

LAYER	TECHNOLOGY	COST	WI
Language	Python 3.9+	Free	Ev
Scheduler	GitHub Actions	Free	Ru
AI Engine	Google Gemini 1.5 Flash	Free tier	Re the
Database	Supabase (PostgreSQL)	Free tier	St
Frontend	Streamlit	Free tier	Th
Feed parsing	feedparser	Free	Re
PDF reading	pdfplumber + httpx	Free	Do lin
Validation	Pydantic v2	Free	En
Retries	tenacity	Free	Au

Why Each Technology?

Why Supabase instead of Firebase or another databa

Compliance data is relational — regulations have jurisdictio

(which Supabase runs on) is the industry standard for relati
alternative) uses a document/NoSQL model that makes filte
also provides a free tier, a clean Python client library, and a
query directly. Future features like linking events to tags or
in NoSQL.

Why Google Gemini instead of OpenAI GPT-4?

Two reasons: cost and context window. Gemini 1.5 Flash ha
regulatory PDFs in a single call. More importantly, it offers a
million tokens per month). GPT-4o is too expensive for a ze
require dedicated compute and cannot run on GitHub Actio

Why GitHub Actions instead of a cloud server?

A dedicated server costs money every hour, even when the
to run once a day for about 5 minutes. GitHub Actions prov
public repos). Five minutes per day is 150 minutes per mon
built-in logging, failure email notifications, and manual trigg

Why Streamlit instead of React?

The entire project is Python. Streamlit lets a Python develo
pure Python — no HTML, CSS, or JavaScript required. It de
pointing it at a GitHub repository. If the product grows, the
without touching any backend code, because the UI only re

Why Pydantic for validation?

AI models are non-deterministic. The same prompt sent to
different structures. Pydantic acts as a strict gatekeeper: if
expected data shape, Pydantic throws an error before anyth
data from ever reaching production.

---

models.py — The Data Contract

Location: `src/models.py`

This is the most important file in the project. It defines the e
must take before it can enter the database. Think of it as th
all must agree to.

`ComplianceEvent`

A Pydantic model — a Python class that enforces strict data
must produce an object matching this model before it is sav
rejected.

- **title** (text, required) — The headline of the regulation or e
  source.
- **source_url** (URL, required) — A validated URL pointing t
  never allowed to generate this field — it is always overrid
  RSS feed.
- **jurisdiction** (text, required) — The primary country or reg
  "Estonia").
- **status** (one of four values, required) — Must be exactly c
  or `Guidance`. Pydantic rejects any other value.
- **effective_date** (date or null, optional) — When the rule b
  mention a date, this must be null — the AI is explicitly ins
- **relevance_score** (integer 1–10, required) — How importa
  range; any other number is rejected.
- **summary** (text, required) — A concise, two-sentence pla
  and why it matters.
- **recommended_action** (text, required) — A specific next
  language.

CRITICAL RULE

The `source_url` field exists in the model for database sto
The Python pipeline always overwrites whatever the AI retu
RSS feed. This prevents URL hallucination.

---

feeds.py — The Data Collector

Location: `src/ingestion/feeds.py`

This file has one job: given an RSS feed URL, go fetch all th
handles both regular web articles and PDF documents.

`fetch_rss_feed(feed_url)`

Takes the URL of an RSS feed and returns a list of article dic

- Uses `feedparser` to download and parse the RSS XML
- If the feed fails to download, catches the error, logs a wa continues to the next feed rather than crashing.
- If the feed XML is malformed but has no entries, also retu
- For each article entry, extracts the link URL, title, and pul
- **PDF detection:** If the article link ends in `.pdf`, calls `ex` the PDF. If no text is extractable, the article is skipped.
- For regular web articles, looks for full HTML content in th summary field.
- Returns a list of dictionaries: each containing `title`, `l`

`extract_text_from_pdf(pdf_url)`

Some regulatory bodies publish updates as PDF files. This f

- Uses `httpx` to download the PDF from its URL, with a 3 connections.
- Opens the downloaded bytes in memory (without saving
- Iterates through every page and extracts text from each.
- Joins all pages with newline characters and returns the c
- If anything fails (network error, corrupted PDF, password returns an empty string — which causes the article to be

pipeline.py — The Orchestrator

Location: `src/ingestion/pipeline.py`

This is the main script that ties everything together. When (file. It coordinates all other modules: fetching feeds, checki the database.

The six monitored sources

REGULATOR
FATF (Financial Action Task Force)
FinCEN (Financial Crimes Enforcement Network)
OFAC (Office of Foreign Assets Control)
FCA (Financial Conduct Authority)
AMLA (Anti-Money Laundering Authority)
EBA (European Banking Authority)

`run_pipeline()`

The main function that runs the entire ingestion process. It p
article within each feed one at a time. Returns a summary di

Step-by-step flow:

1. Initialises a stats counter: `found` (total articles seen), `
   (successfully saved), `errors` (failures).
2. Loops through each of the six feed URLs.
3. For each feed, calls `fetch_rss_feed()` to get the list o
4. For each article:
   - Increments `found`.
   - Skips any article with no URL (logs a warning, increme
   - Calls `check_exists(url)` to check if this URL is alre
     duplicate.
   - Checks that raw text is not empty. If it is, skip it.
   - Calls `process_article()` from processor.py with th
   - Calls `insert_event()` to save the validated event to
   - Increments `inserted` and logs the title.
   - **Sleeps for 4.5 seconds** before the next article (rate li
     per minute).
5. If any single article throws an exception, catches it, logs
   continues. One bad article never crashes the whole pipel
6. After all feeds finish, logs the final summary and returns

> **WHY 4.5 SECONDS BETWEEN ARTICLES?**
>
> Google Gemini's free tier allows a maximum of 15 API reque
> each call keeps the pipeline at roughly 13 requests per minu

---

processor.py — The AI Brain

Location: `src/ai/processor.py`

This file contains the logic for communicating with Google (
returns a fully validated `ComplianceEvent` object. The AI n
data that must pass Pydantic validation first.

How the AI prompt works

The processor builds a prompt that:

- Provides the full text of the regulatory article.
- Instructs Gemini to act as a compliance analyst and extr
- Provides the exact scoring rubric (1–10) so relevance is
- Gives the four allowed status values with their definition
- Explicitly instructs: *"If the text does not contain enough return null. Do not guess."*
- Tells the AI to output strict JSON matching the required

```
process_article(raw_text, url)
```

Takes the raw text of a regulatory article and its verified URL
the database.

- Initialises the Google Gemini client using the `GEMINI_AP`
- Constructs a detailed prompt combining the article text a
- Calls the Gemini API with the `response_schema` parame forcing the model to return valid JSON matching that sch
- Parses the JSON response and instantiates a `Complianc` malformed or missing required fields, Pydantic raises a `
- **URL override:** After receiving the AI's output, Python imp the verified `url` argument passed into the function. Th
- Returns the validated `ComplianceEvent` object

- Returns the validated `ComplianceEvent` object.

> **THE URL OVERRIDE PATTERN — WHY IT MATTERS**
>
> AI models sometimes hallucinate URLs — they invent plausi
> already have the verified, real URL from the RSS feed, there
> Python code always discards the AI's URL and replaces it w
> record in the database links to a real page.

---

database.py — The Data Layer

Location: `src/database.py`

This file handles all communication with Supabase. It is the
No other file forms database queries directly — everything
database logic centralised and easy to change.

`_get_secret(key)`

A private helper function (the underscore prefix signals inte
environment.

- First tries `st.secrets` — the Streamlit Cloud secrets m
  Cloud in production.
- If Streamlit is not available (e.g., during GitHub Actions),
  environment variables or a local `.env` file.
- This dual-mode approach means the exact same code w
  production without changes.

`get_supabase_client()`

Creates and returns a Supabase client connection. Called at

- Calls `_get_secret()` to retrieve `SUPABASE_URL` and `
- If either is missing, raises an `EnvironmentError` with a
  This prevents cryptic errors.
- Returns a live Supabase client object ready for querying.

`check_exists(url)`

Checks whether an article with a given URL already exists in

- Queries `compliance_events` for any row where `sourc`
- Only selects the `id` column — no need to retrieve all da
- Returns `True` if at least one matching row is found, `Fa`
- If the query fails, catches the error, logs it, and re-raises error without crashing.

`insert_event(event)`

Saves a validated `ComplianceEvent` to the database. The

- Accepts a `ComplianceEvent` Pydantic object. At this po guaranteed correct.
- Calls `event.model_dump(mode="json")` to convert the with JSON-compatible values (e.g., dates become ISO-fo
- Inserts the row into `compliance_events` via the Supaba
- Returns the newly created database row, including the au timestamp.
- If the insert fails, catches and re-raises the error. The pip

app.py — The Dashboard

Location: `src/ui/app.py`

The user-facing web application. It reads data from Supaba never writes to the database — it is purely read-only. The e into a live web page.

`fetch_events()`

Fetches all compliance events from Supabase and returns th data structure in memory).

- Decorated with `@st.cache_data(ttl=300)` — Streamlit

page within that window serves data from cache, reducir fast.

- Queries all columns from `compliance_events`, sorted b
- If no data is returned, returns an empty DataFrame.
- Converts `effective_date` to Python date objects (mal `errors="coerce"`).
- Converts `created_at` to datetime objects for proper di

`main()`

Builds and renders the entire dashboard page. Streamlit call web page.

- **Page config:** Sets the browser tab title, icon to 📡, and l
- **Header:** Renders the title and a disclaimer stating the da legal advice.
- **Sidebar filters:**
  - *Minimum Relevance Score slider:* Draggable from 1 to 1 (show everything).
  - *Jurisdiction multi-select:* Shows all unique jurisdictior about. Defaults to all selected.
  - *Status multi-select:* Shows all unique statuses. Same
  - *Enterprise teaser panel:* A marketing section listing pa commercial interest.
- **Data fetching:** Calls `fetch_events()`. If it throws an er result is empty, displays an info message and stops.
- **Filter application:** Applies each sidebar filter to the Data jurisdictions, then statuses — to produce the final `filte`
- **Summary metrics:** Three headline numbers in a row — t distinct jurisdictions. Updates instantly when filters chan
- **Data table:** Renders the filtered data with custom colum bar, effective date as a clean date, and source URL as a

Database Schema

One table in the database: `compliance_events`. Every artic here.

COLUMN	TYPE	REQUIRED?
id	UUID	Auto
source_url	TEXT (unique)	Yes
title	TEXT	Yes
raw_text	TEXT	No
jurisdiction	TEXT	Yes
status	TEXT	Yes
effective_date	DATE	No
relevance_score	INTEGER (1–10)	Yes
summary	TEXT	Yes
recommended_action	TEXT	Yes
created_at	TIMESTAMP	Auto

AI Scoring Rules

The AI follows a strict rubric for relevance scoring and statu every prompt sent to Gemini.

Relevance Score (1–10)

SCORE	LEVEL	EXAMPLES
10	Critical	FATF Recommendations updated; maj action affecting all institutions globall
8–9	High	EU's Anti-Money Laundering Regulati high-profile fines against tier-1 banks.
5–7	Medium	Jurisdiction-specific guidance; FCA "I typologies from an FIU.
1–4	Low	Niche local regulations; routine speec immediate compliance action requirec

Status Classification

STATUS	WHEN TO USE IT
Proposed	Draft legislation, consultation papers, or rule still accepting feedback.
In Force	Final rules, acts, or directives that are currer
Enforcement	Fines, settlements, prosecutions, regulatory institution or individual.
Guidance	Advisory notices, typology reports, FAQs, "D clarifies expectations without creating new l

---

Daily Automation

File: `.github/workflows/daily_run.yml`

This YAML file tells GitHub Actions how to run the pipeline a executes it on a schedule without any human intervention.

What happens when GitHub Actions fires

1. **Trigger:** The workflow fires at 6:00 AM UTC daily (`cron` manually from the GitHub web interface.
2. **Environment setup:** GitHub spins up a fresh Ubuntu Lin

3. **Secrets injection:** GitHub reads the three secrets from t
   `SUPABASE_KEY`, `GEMINI_API_KEY`) and makes them ava
   visible in logs.
4. **Code checkout:** Downloads the latest code from the `ma`
5. **Python setup:** Installs Python 3.9.
6. **Dependency installation:** Runs `pip install -r requir`
7. **Pipeline execution:** Runs `python -m src.ingestion.pip`
8. **Completion:** The virtual machine is destroyed. All logs a

> **FAILURE NOTIFICATIONS**
>
> If the workflow fails (e.g., Gemini is down, Supabase is unre
> notification to the repository owner with the full error outpu

---

Secrets & Security

The three secrets

SECRET NAME	WHAT IT IS
SUPABASE_URL	The URL of the Supabase project
SUPABASE_KEY	The Supabase API key
GEMINI_API_KEY	The Google Gemini API key

How secrets are kept safe

- Secrets are **never hardcoded** in the source code. The c
  `os.getenv()` or `st.secrets`.
- The `.env` file used for local development is listed in `.g`
  repository.
- In production (GitHub Actions), secrets are stored in Gitl
  only.
- In production (Streamlit Cloud), secrets are stored in Str
  `st.secrets`.

Database access levels

- **Service role key:** Full read and write access. Used by th
  Never exposed to the UI.
- **Anon key:** Read-only access to `compliance_events` (er
  by the Streamlit UI. Even if someone extracted this key, t
  modify or delete anything.

> **NO PERSONAL DATA**
>
> The system only processes publicly available regulatory tex
> data, no proprietary bank information, and no personally id
> system.

Deployment

Local development

A developer running the project locally needs Python 3.9+,
`requirements.txt`), and a `.env` file with the three secrets
`python -m src.ingestion.pipeline` or start the dashboar

Production — Backend (GitHub Actions)

The pipeline runs automatically at 6 AM UTC. The only ong
Secrets remain valid. If a secret expires or is rotated, updat
Actions.

Production — Frontend (Streamlit Community Cloud

The dashboard is deployed by linking the GitHub repository
`main` triggers an automatic redeploy. The `SUPABASE_URL`
Streamlit's Secrets Manager in the app settings.

Rollback

- **UI rollback:** Revert the commit in GitHub. Streamlit auto
- **Pipeline rollback:** Revert the commit. The next cron run
  restarts are needed.

Engineering Rules

These non-negotiable rules apply to every code change ma

RULE 1 — PYDANTIC IS THE FIREWALL

Every AI response must be validated against the `Complian` touches the database. The LLM is non-deterministic; never

RULE 2 — THE AI NEVER GENERATES THE SOURCE URL

The `source_url` must always come from the verified RSS the LLM responds. This prevents hallucinated URLs from er

RULE 3 — ZERO-COST MANDATE

Do not introduce paid APIs, paid databases, or paid hosting covers the project's usage. If a suggested technology has r

RULE 4 — NO HARDCODED SECRETS

API keys and database credentials must never appear in so pipeline and `st.secrets` for the UI. The `.env` file must

RULE 5 — RESILIENCE ON EXTERNAL CALLS

All calls to Gemini, Supabase, and RSS feeds must be wrap never crash because one article or one feed failed. Use `te`

RULE 6 — STRICT SEPARATION OF CONCERNS

The UI only reads from the database. The pipeline only writ call each other's functions. Keep ingestion logic in `src/in` `src/ui/`, and database logic in `src/database.py`.

Future Roadmap

The current system is the MVP — a working proof of concep versions.

Near-term (V1.1)

- **User authentication:** Supabase Auth so multiple users c
- **Daily email digest:** A GitHub Actions job that sends a fo new events.
- **Slack / Teams alerts:** Webhook-based notifications for c real time.

Medium-term (V2.0)

- **Custom RSS feeds:** Allow users to add their own regula
- **Audit log:** A separate table recording every pipeline run viewable from an admin section.
- **Tag system:** Allow teams to tag events (e.g., "Action Re tags.

Enterprise edition (V3.0)

- **Multi-tenancy:** Support multiple separate organisations
- **Automated report generation:** One-click export of a cu
- **Advanced AI analysis:** Cross-regulation impact analysis our existing policy on correspondent banking?").